ADANI PUBLIC SCHOOL
MUNDRA

INFORMATICS PRACTICES | CODE 065

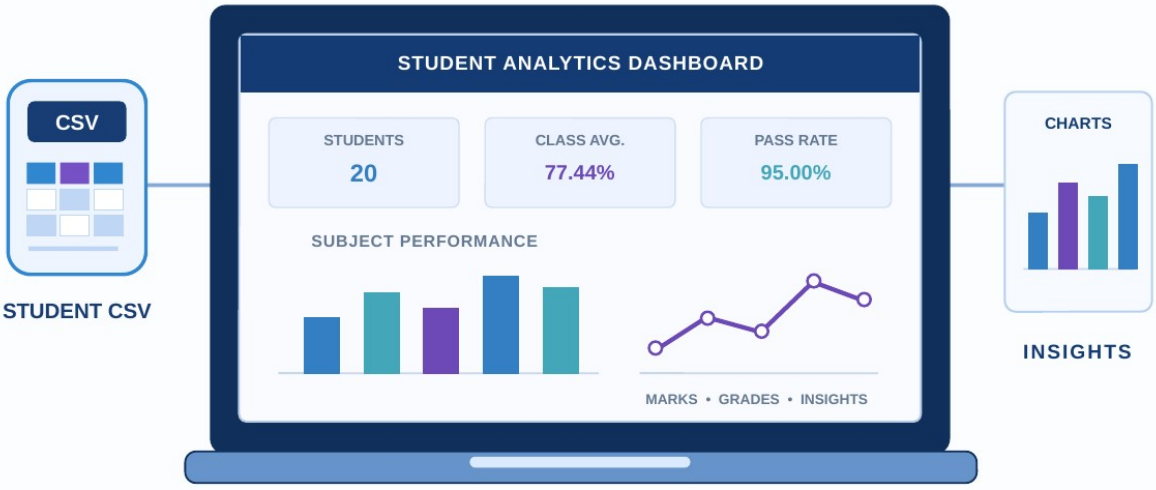
IP PROJECT

STUDENT MANAGEMENT SYSTEM

Student Records, Academic Analytics & Data Visualization

BUILT WITH

PYTHON | CUSTOMTKINTER | PANDAS | MATPLOTLIB | CSV



PROJECT BY

ANSH PATEL

CLASS XII-A | ROLL NO. 06

ACADEMIC SESSION 2026-27

PYTHON • CUSTOMTKINTER • PANDAS • MATPLOTLIB • CSV

adani
Public School
Mundra

CERTIFICATE

OF APPRECIATION

This is to Certify that

Ansh Patel

Ansh Patel, presently studying in Class XII Science at Adani Public School, Mundra, has successfully completed the INFORMATICS PRACTICES PROJECT titled “STUDENT MANAGEMENT SYSTEM” using Python, CustomTkinter, Pandas, Matplotlib and CSV, under the guidance of Ms. Khalida Ma’am during the academic year 2026-2027 as part of the curriculum.

The project report documents the program, source code, sample dataset, outputs and analysis.
The candidate has worked on the project and its documentation under the subject teacher’s guidance.



Khalida Ma'am
Informatics Practices Teacher

Hemant Kumar Sharma
Principal

External Examiner

FORMAL PROJECT DETAILS

Student Management System

This report documents the Class XII Informatics Practices project for the academic session 2026-27. It describes the application, its data model, implemented operations, sample analysis, outputs, test cases and complete source code.

Item	Details
Project title	Student Management System
Project subtitle	Student Records, Academic Analytics and Data Visualization
Student name	Ansh Patel
Class / section	XII-A
Roll number	06
Subject	Informatics Practices (Code 065)
School	Adani Public School, Mundra
Academic session	2026-27
Project type	Python desktop application; single-user and offline
Main source file	Project Program.py (539 source lines)
Data file created by the application	students.csv (13 columns)
Demonstration dataset	Student.csv (20 artificial records)
Technologies	Python, Tkinter, CustomTkinter, Pandas, Matplotlib and CSV storage
Main functions	Record management, search, CSV import, validation, calculations, analytics and charts

> Dataset note: the twenty student records used in the examples are artificial demonstration records. They do not describe real students and are included only to demonstrate the application.

Abstract

The Student Management System is a Python desktop application designed to store and manage student academic records in a CSV file. It provides a graphical form for entering student details and marks, a table for viewing records, operations for adding, updating and deleting entries, and a CSV import feature.

For each record, the application calculates total marks, percentage and grade from four subject marks: English, Physics, Chemistry and Mathematics. It validates required fields and numeric ranges before saving data. Pandas is used to read, filter, update and analyse the records, while Matplotlib draws three charts: top five students, average marks by subject and grade distribution.

The dashboard presents four indicators: number of students, average percentage, topper name and pass percentage. Ten analytics operations provide class averages, topper and lowest scorer information, top and bottom five lists, subject averages, pass/fail counts, pass percentage and grade distribution. The application is intended for one user working offline; the CSV file is its only persistent storage layer.

Project snapshot	Value
Source code lines	539
Functions	15
Data columns	13
Demonstration records	20
Dashboard indicators	4
Analytics operations	10
Chart types	3
Sample average percentage	77.44%
Sample pass percentage	95.00%

Contents

The report is arranged in the same sequence as the project workflow: purpose, implementation, data, results, testing and appendices.

No.	Section	Page
1	Introduction, problem statement and scope	6
2	Objectives	7
3	Technology stack	7
4	System requirements and setup	8
5	System architecture and data flow	9
6	Data model and field reference	10
7	Calculation and grading logic	11
8	Feature matrix and dashboard	11
9	Record operations and CSV import	12
10	Search and automatic calculations	13
11	Analytics and graphical visualisation	13
12	User interface design	15
13	Operational workflows	16
14	Function reference	17
15	Program flow and event handling	17
16	Input validation	18
17	Analytics pipeline	18
18	Dataset documentation	19
19	Complete dataset: Student.csv	20
20	Results computed from the sample dataset	21
21	Application outputs	22
22	Test cases	23
23	Limitations and future enhancements	24
24	Conclusion	24
25	Learning outcomes	25
26	Bibliography	25
A	Appendix A: Complete source code	26
B	Appendix B: Data file reference	34

Appendix A contains the complete source-code listing. Appendix B summarises the fields and validation rules used by the CSV data file.

1. Introduction

1.1 Background

Schools maintain student details and academic marks for routine administration and performance review. When records are kept manually or spread across separate files, finding an entry, recalculating totals and preparing summaries can take time and may lead to inconsistent values.

A computerised record system stores information in a consistent structure and calculates derived values automatically. In this project, the record table, dashboard, analytics and charts are based on the records stored in one CSV file.

1.2 Problem statement

The project addresses five practical difficulties: repeated arithmetic, slow searching, updates that leave old totals unchanged, time-consuming preparation of class statistics and difficulty comparing results from a long list of marks.

Manual difficulty	Effect	Project response
Repeated arithmetic	Totals and percentages must be recalculated by hand.	Calculate total, percentage and grade when a record is saved.
Slow searching	A register must be scanned to find a student.	Search by name or roll number.
Inconsistent updates	A changed mark may not be reflected in the total.	Recalculate derived fields on save.
Slow reporting	Class summaries require repeated calculations.	Provide ten built-in analytics operations.
Hard-to-see patterns	Comparing many values is difficult.	Draw three charts from the stored dataset.
Incorrect entries	Duplicate rolls or out-of-range marks may be stored.	Validate form input and imported CSV records.

1.3 Proposed solution

The proposed solution is a single-window desktop application built with Python, CustomTkinter, Tkinter, Pandas and Matplotlib. The form collects student information, validation checks the values, calculated fields are added, and Pandas writes the record to students.csv. The table and dashboard refresh after operations, while analytics and chart buttons read the current stored records.

> Form or CSV import → validation → total / percentage / grade → students.csv → record table, dashboard, analytics and charts

1.4 Scope and intended users

The application is intended for a single operator, such as a teacher or a student demonstrating the project. It works offline and does not require a database server or network connection.

Included in the present implementation	Outside the present scope
Store student records in one CSV file with thirteen columns.	Attendance, fees, exam scheduling and other school modules.
Add, update and delete records.	User accounts, login screens and permission levels.
Import a CSV file after validation.	Multi-user access, cloud storage and a server.

Included in the present implementation	Outside the present scope
Search by name or roll number and restore the full list.	Relational database storage.
Calculate totals, percentages and grades; show summaries and charts.	Exportable report cards and printable reports from the application.

2. Objectives

The objectives connect the user-facing features to the calculations and reports produced by the application.

No.	Objective	Description
1	Record storage	Maintain student data in a structured CSV file.
2	Add student	Accept a new record through a validated form.
3	Update student	Modify a stored record identified by roll number.
4	Delete student	Remove a record after confirmation.
5	CSV import	Read an external file and validate columns and values.
6	Search	Find records by name or roll number.
7	Automatic total	Sum marks from four subjects.
8	Automatic percentage	Divide total marks by four.
9	Automatic grade	Assign a grade using fixed percentage thresholds.
10	Dashboard	Show four indicators that refresh after table updates.
11	Analytics	Generate ten summaries from the stored records.
12	Visualisation	Display three Matplotlib charts.

3. Technology Stack

The program imports six modules. The technologies below are listed according to their role in the submitted implementation.

Technology	Role in the project
Python	Main programming language; functions, conditions, loops, arithmetic, dictionaries and error handling.
Tkinter	Standard GUI toolkit; StringVar values, Treeview table, scrollbars, message dialogs and file chooser access.
CustomTkinter	Window and modern widgets, form controls, buttons, dark appearance and blue theme.
Pandas	Reads and writes CSV data, builds DataFrames, appends and updates rows, filters records and calculates summaries.

Technology	Role in the project
Matplotlib	Creates the top-five bar chart, subject-average bar chart and grade-distribution pie chart.
CSV file	The persistent data store. The program reads and writes the local students.csv file.

Main imports from Project Program.py:

```
import os
import tkinter as tk
from tkinter import ttk, messagebox
import customtkinter as ctk
import pandas as pd
import matplotlib.pyplot as plt
```

4. System Requirements and Setup

The values below are recommended working requirements for a small desktop application. They are guidance, not a claim that the program has been benchmarked on every listed configuration.

Component	Recommended requirement	Reason
Processor	Modern dual-core processor	The program performs ordinary arithmetic and table operations.
Memory	4 GB RAM or more	Allows Python, Pandas, Matplotlib and the GUI to run together.
Disk space	About 500 MB for Python and packages, plus data files	The CSV files are small; most space is used by Python and installed packages.
Display	1366 × 768 or larger	The main window is configured to 1400 × 800 pixels.
Input	Keyboard and mouse	Used to enter values and operate buttons.
Operating system	Windows 10/11 or a desktop Linux environment supported by Python	The program needs Python and a working Tk/Tcl graphical environment.

4.1 Required software

- Python 3.9 or later is recommended. Tkinter is included with many Python installations; Linux systems may require the distribution package for Tk support.
- The external packages used by the program are CustomTkinter, Pandas and Matplotlib.

4.2 Installation and running

```
python -m pip install customtkinter pandas matplotlib
python "Project Program.py"
```

4.3 Files and folders

File	Purpose
Project Program.py	The Python program that builds the interface and implements operations.
students.csv	The main data file. The program creates it with the thirteen column headings if it is missing.
Student.csv	The twenty-record demonstration dataset included with this report; import it through the Import CSV button.

5. System Architecture

The application can be understood as six connected layers. The user works with the interface; each requested operation validates data where required, uses Pandas to access the records and returns the result to the table, dashboard, analytics dialog or chart window.

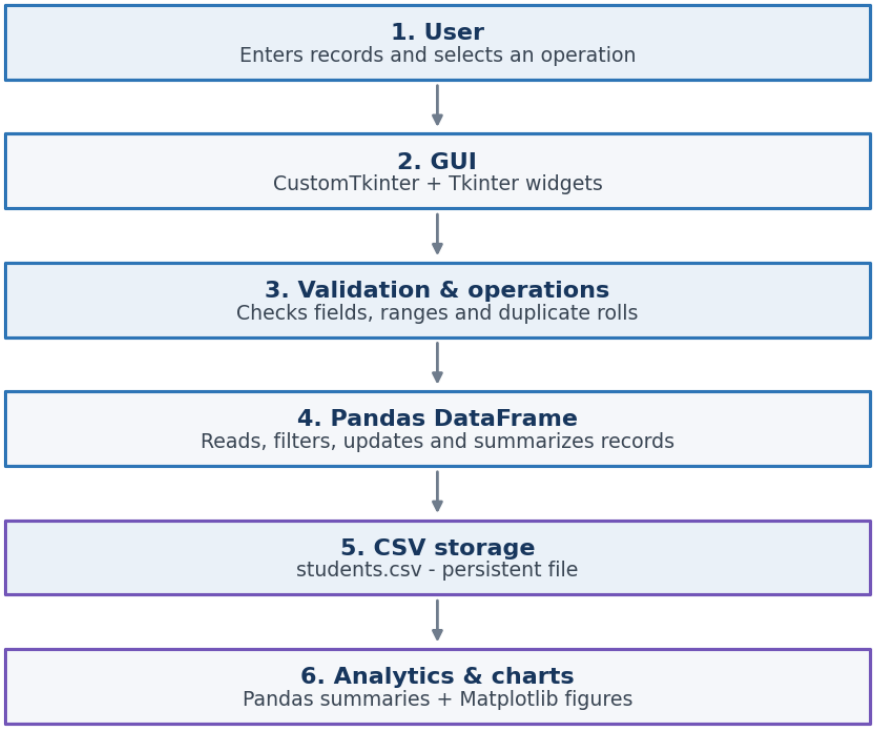


Figure 1. Architecture flow based on the submitted implementation.

Layer	Component	Responsibility
1	User	Enters values, selects buttons and reviews results.
2	CustomTkinter / Tkinter GUI	Displays the form, dashboard, record table, dialogs and chart controls.
3	Validation and operations	Checks values and performs add, update, delete, import and search operations.
4	Pandas DataFrame	Provides the working tabular representation of records.
5	students.csv	Stores records between application runs.
6	Analytics and charts	Uses Pandas expressions and Matplotlib to present summaries.

5.1 One data path, several views

The table and dashboard are rebuilt from the stored data after record changes. Analytics and charts read the file when they are requested. This keeps reporting tied to the same data source. The search view is a filtered view of the table, whereas the analytics buttons read the complete stored file.

5.2 What is not part of the architecture

The submitted code does not implement a cloud database, server, web API, login system or network connection. The CSV file is the only persistent storage layer.

6. Data Model and Field Reference

The application uses a single CSV file named students.csv. It contains one header row and one row per student record. The COLUMNS list in the source file defines the order of the thirteen fields.

Field	Type	Source	Rule / meaning
Roll Number	Integer	User-entered	Required; must be unique.
Name	Text	User-entered	Required; must not be blank.
Class	Text / choice	Combo box	11 or 12.
Section	Text / choice	Combo box	A, B, C or D.
Gender	Text / choice	Combo box	Male, Female or Other.
Age	Integer	User-entered	Greater than 0 and no more than 100.
English	Number	User-entered	From 0 to 100.
Physics	Number	User-entered	From 0 to 100.
Chemistry	Number	User-entered	From 0 to 100.
Mathematics	Number	User-entered	From 0 to 100.
Total Marks	Number	Calculated	Sum of four subject marks.
Percentage	Decimal	Calculated	Total marks divided by four.
Grade	Text	Calculated	Assigned by calculate_grade().

6.1 One record in full

Field	Value
Roll Number	101
Name	Aarav Sharma
Class / Section	12-A
Gender / Age	Male / 17
English	85
Physics	92
Chemistry	88
Mathematics	90
Total Marks	355
Percentage	88.75%
Grade	A

This example shows that the first ten fields are supplied through the form or CSV import, while Total Marks, Percentage and Grade are produced by the program.

7. Calculation and Grading Logic

> Total Marks = English + Physics + Chemistry + Mathematics
> Percentage = Total Marks ÷ 4
> Grade = calculate_grade(Percentage)

The program calculates three derived values when a record is saved. The grade function checks thresholds from highest to lowest so that a percentage of exactly 90 receives A+ rather than A.

Grade	Percentage condition	Interpretation	Sample count
A+	90 or above	Outstanding	4
A	80 to less than 90	Very good	8
B	70 to less than 80	Good	2
C	60 to less than 70	Satisfactory	2
D	40 to less than 60	Pass	3
F	Below 40	Fail	1

Table 7. Grade thresholds implemented by calculate_grade() and the distribution in the demonstration dataset.

7.1 Worked examples

Roll	Name	Marks added	Total	Percentage	Grade
101	Aarav Sharma	85 + 92 + 88 + 90	355	88.75%	A
106	Sneha Reddy	95 + 93 + 96 + 94	378	94.50%	A+
117	Nikhil Bansal	40 + 38 + 42 + 35	155	38.75%	F
120	Riya Malhotra	93 + 91 + 94 + 92	370	92.50%	A+

7.2 How a calculated value reaches the file

When Add or Update is pressed, get_form_data() reads the editable fields, checks the values, calculates total marks, percentage and grade, and returns a dictionary with thirteen values. The corresponding operation writes the row through Pandas, then the table and dashboard refresh.

8. Feature Matrix and Dashboard

Feature	Function(s)	What it does
Dashboard cards	create_kpi_card(), update_dashboard()	Four live indicators: count, average, topper, pass percentage.
Add Student	get_form_data(), add_student()	Validates and appends a new student record.
Update Student	fill_form_from_tree(), update_student()	Loads selected values and replaces the matching record.
Delete Student	delete_student()	Requests confirmation before removing a record.
Import CSV	import_csv()	Checks an external dataset before appending its rows.
Search	search_student()	Filters by student name or roll number.

Feature	Function(s)	What it does
View All	refresh_table()	Clears the search filter and reloads all stored records.
Automatic calculations	get_form_data(), calculate_grade()	Computes total marks, percentage and grade.
Analytics	show_analytics(action)	Shows ten data summaries in message boxes.
Charts	plot_graph(graph_type)	Opens three Matplotlib charts.

8.1 Dashboard indicators

Card	Sample value	Meaning
Total Students	20	Number of rows in the dataset.
Average Percentage	77.44%	Mean of the Percentage column.
Topper Name	Sneha Reddy	Name with the highest percentage.
Pass Percentage	95.00%	Share of records whose grade is not F.

When the stored file has no rows, the dashboard displays zero values and N/A for the topper rather than trying to select a highest-scoring record.

9. Record Operations and CSV Import

9.1 Add a student

1. Fill the ten editable fields in the student form.
2. Press Add. The program checks required fields, converts numeric entries, checks age and marks ranges, and checks whether the roll number is already stored.
3. If validation succeeds, the program appends the record and refreshes the table and dashboard.

9.2 Update a student

1. Select a record in the table to copy its values into the form, or type its roll number.
2. Edit the required values and press Update.
3. The program verifies that the roll number exists, recomputes derived fields and writes the revised row.

9.3 Delete a student

1. Enter the roll number for deletion.
2. The program checks that the roll number is valid and stored.
3. A confirmation dialog asks whether the record should be deleted. Choosing No leaves the file unchanged; choosing Yes removes the matching row and refreshes the display.

9.4 Import a CSV file

Import CSV opens a file chooser and reads an external CSV file with Pandas. Before the incoming records are appended, the program checks that required columns exist, numeric values can be converted, values are not missing, and imported roll numbers do not duplicate rolls already in the stored file.

Import check	Purpose	Failure behaviour
Required columns	Confirms that the file contains all thirteen required headings.	Shows a missing-columns error.
Numeric values	Checks roll number, age and subject marks.	Rejects rows containing invalid required values.
Missing values	Checks for empty required cells.	Rejects the import before the accepted dataset is written.
Duplicate roll numbers	Ensures imported rolls are not already stored.	Shows the duplicate roll numbers and stops the import.

10. Search, View All and Automatic Calculations

Operation	Implemented behaviour
Search	Converts the query to lower case and matches it against the name and roll number.
Empty search query	Reloads the full data through refresh_table().
View All	Clears the query and rebuilds the table from the file.
Total Marks	Adds the four subject marks when a record is saved.
Percentage	Divides total marks by four.
Grade	Calls calculate_grade() using the computed percentage.

Only ten of the thirteen fields are editable. Total Marks, Percentage and Grade are calculated by the program so that a corrected mark does not leave derived values stale.

11. Analytics and Graphical Visualisation

Each analytics button supplies its caption to show_analytics(action), which selects a Pandas expression and displays the formatted result. The three graph buttons call plot_graph(graph_type) and create a separate Matplotlib window.

Analytics button	Result produced
Class Average	Average percentage grouped by class.
Topper	Student with the highest percentage.
Lowest Scorer	Student with the lowest percentage.
Top 5	Five rows with the highest percentage.
Bottom 5	Five rows with the lowest percentage.
Subject Avg	Mean of English, Physics, Chemistry and Mathematics marks.
Pass Count	Number of records with a grade other than F.
Fail Count	Number of records with grade F.
Pass Pct	Pass count divided by total records, multiplied by 100.
Grade Dist	Count of records in each grade.

11.1 The three charts

The charts offer a visual summary of the same twenty-record dataset. The figures below were generated from the sample records printed in this report; the application draws its own versions when the corresponding chart button is pressed.

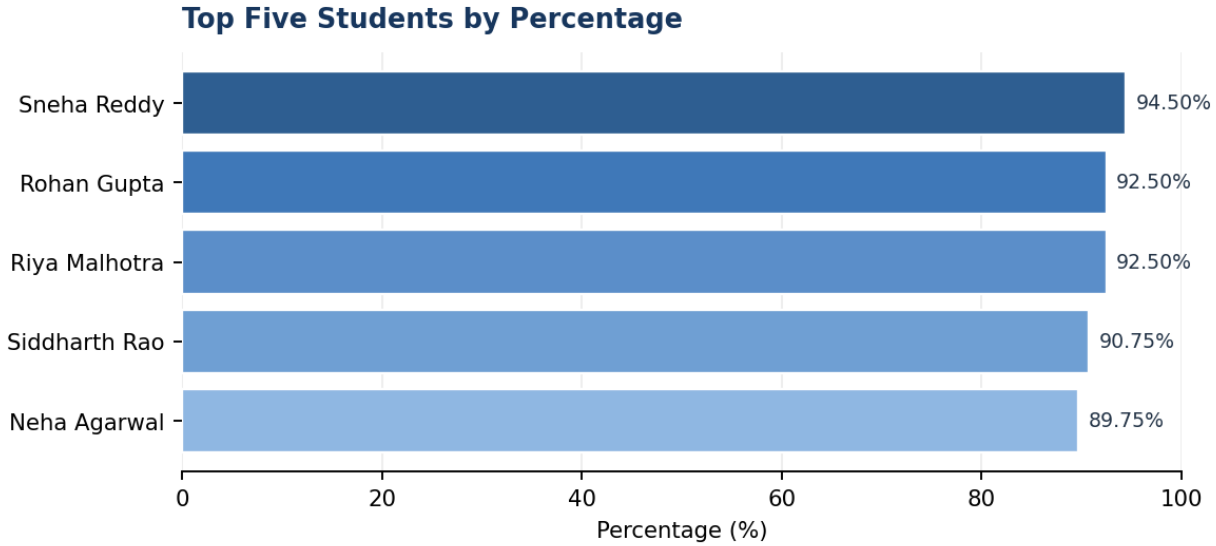


Figure 2. Top five students by percentage.

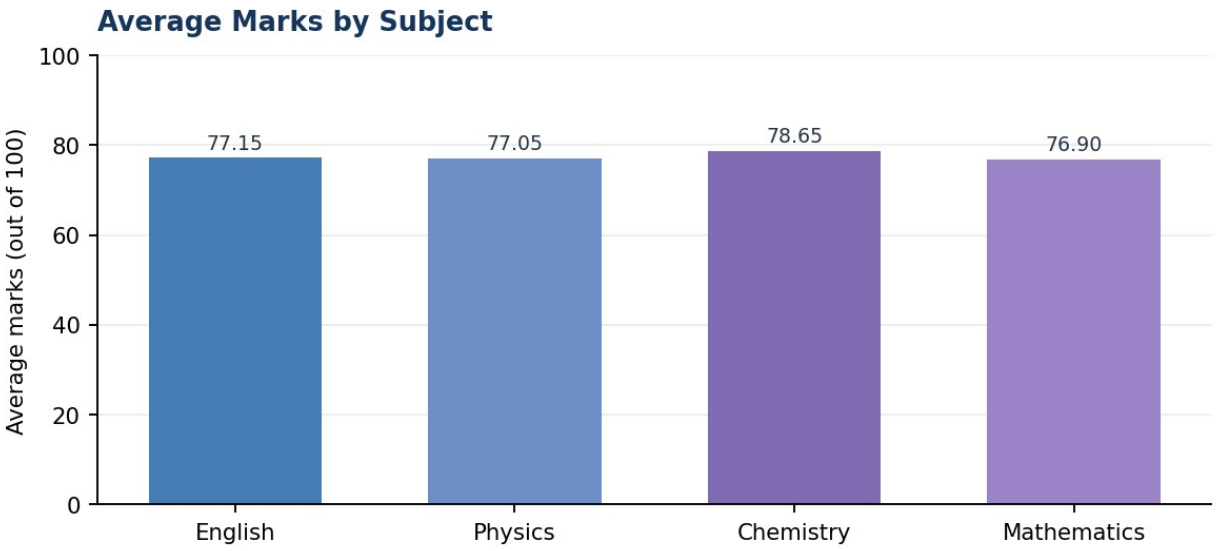


Figure 3. Average marks by subject.

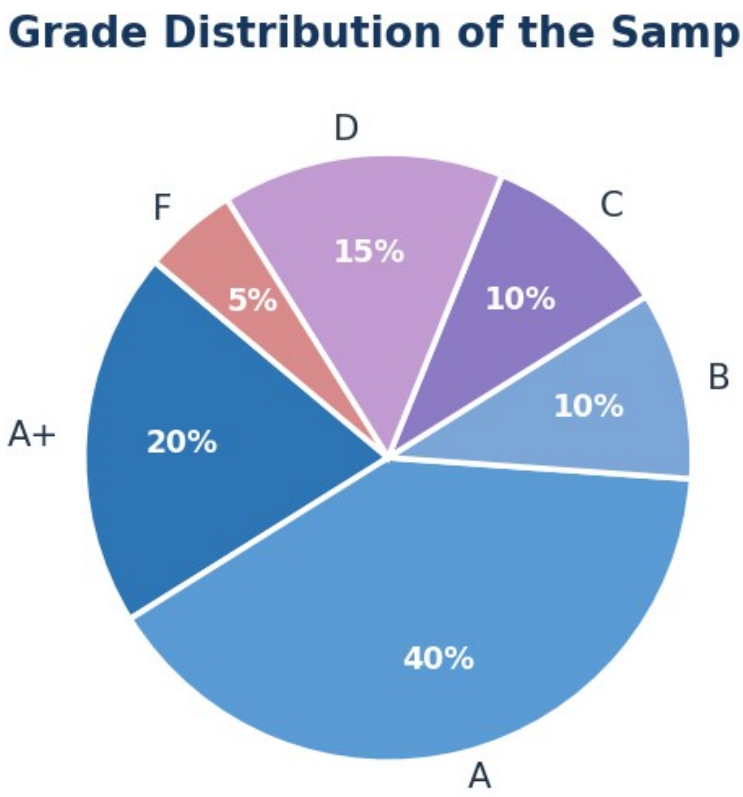


Figure 4. Grade distribution in the sample dataset.

12. User Interface Design

The program opens one window configured to 1400 × 800 pixels. The dashboard, data-entry form, record table, search controls, analytics buttons and chart buttons are all in the same window. The user does not need to switch between multiple screens to manage and analyse records.

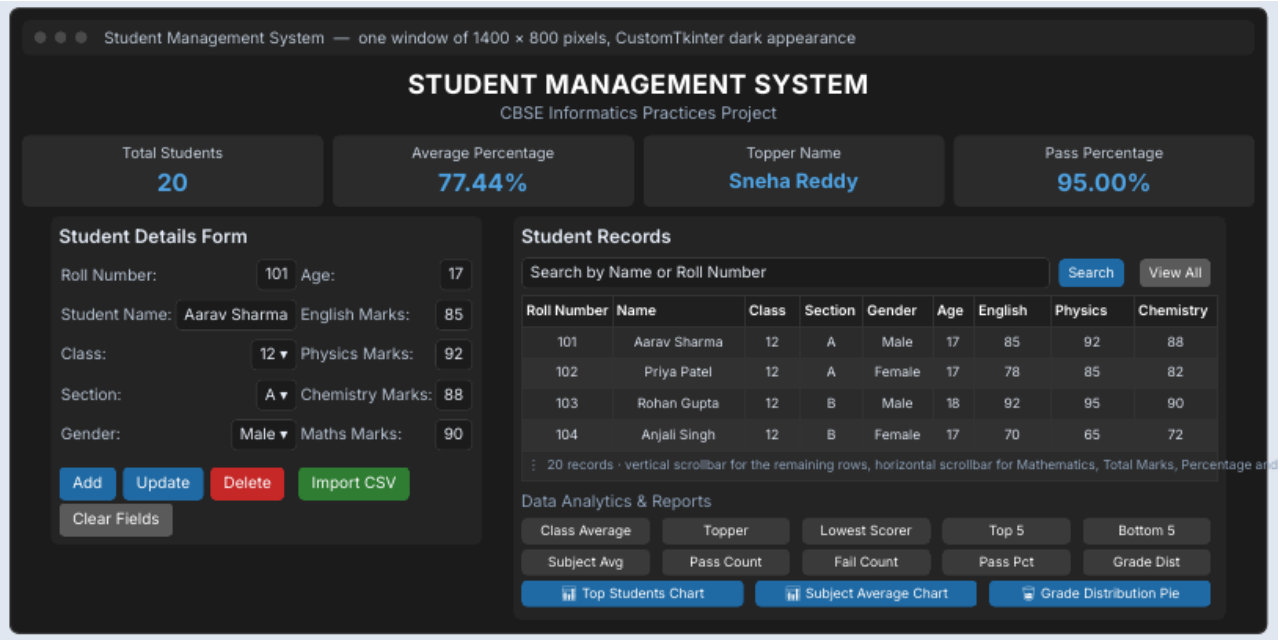


Figure 5. Illustrative interface reconstruction based on the submitted widget names and layout; it is not a direct screen capture of a running session.

Interface region	Implementation detail
Header	Two centred labels for the application name and project subtitle.
KPI row	Four cards created by create_kpi_card(); values use bold accent-colour text.
Student form	Panel containing ten input controls; Class, Section and Gender are combo boxes.
Form buttons	Add, Update, Delete, Import CSV and Clear Fields.
Search bar	Entry field with Search and View All controls.
Record table	ttk.Treeview with thirteen columns plus horizontal and vertical scrollbars.
Analytics grid	Ten operation buttons arranged in a grid.
Chart buttons	Three controls that open the top students, subject average and grade distribution charts.

12.1 Appearance and colours

The code sets CustomTkinter appearance mode to Dark and the default colour theme to blue. The accent colour #1f6aa5 is used for KPI values and chart buttons; #2b2b2b is used for KPI cards and the table background; #2e7d32 is used for the Import CSV button, while Delete uses red.

13. Operational Workflows

13.1 Record management

1. User presses Add, Update or Delete.
2. The program reads the required values and stored records.
3. It checks required fields, numeric values, ranges and record identity.
4. The program appends, replaces or removes the matching row as appropriate.
5. Pandas writes the revised dataset to students.csv.
6. The record table and dashboard are refreshed.

13.2 CSV import

1. Import CSV opens the file chooser.
2. The selected file is loaded into a DataFrame.
3. The columns and required values are validated.
4. The program rejects the import if a check fails; otherwise, the accepted rows are appended.
5. The updated data are saved and the interface refreshes.

13.3 Search and View All

1. The user types a name or roll number and presses Search.
2. The program selects rows matching the query and displays the filtered view.
3. View All clears the query and restores the complete stored list.

13.4 Failure handling

A failed validation prevents a write operation. An unconfirmed deletion leaves the file unchanged. A failed CSV import is rejected rather than partly applied. If no record matches a search, the table becomes empty until View All is used.

14. Function Reference

Function	Original line range	Purpose
calculate_grade(percentage)	45–57	Returns a grade based on percentage thresholds.
clear_form()	60–70	Resets the ten editable form fields.
refresh_table(search_df=None)	73–89	Clears and rebuilds the record table, then updates the dashboard.
update_dashboard(df)	91–109	Calculates the four KPI values.
get_form_data()	111–166	Reads fields, validates values and returns a dictionary with thirteen fields.
add_student()	173–188	Checks roll-number duplication and appends a student.
update_student()	190–205	Replaces the record matching the entered roll number.
delete_student()	207–226	Requests confirmation and removes the matching record.
import_csv()	227–268	Reads a file, validates it and appends accepted records.
search_student()	269–282	Filters by name or roll number.
fill_form_from_tree(event)	283–304	Copies the selected row into the editable fields.
show_analytics(action)	305–345	Computes and displays one of ten summaries.
plot_graph(graph_type)	346–388	Draws one of three charts.
create_kpi_card(parent, title)	389–422	Creates one dashboard card and returns its value label.
make_entry(parent, label, variable, is_combo=False, combo_values=None)	423–470	Creates a label and an entry or combo-box control.

Line ranges are retained from the previous report's source listing. The listing in Appendix A is reconstructed from that source-code appendix and contains 539 source lines.

15. Program Flow and Event Handling

The program defines its interface and then waits for button clicks or table-selection events. The main execution sequence is:

1. Set CustomTkinter appearance and theme.
2. Define the data-file path and thirteen column names.
3. Create students.csv with a header if the file does not exist.
4. Define helper functions for grades, form handling, table refresh and dashboard calculations.
5. Define record operations, CSV import, search and table-selection logic.
6. Define analytics and chart functions.
7. Build the header, dashboard cards, form, search bar, table and report controls.
8. Call refresh_table() to show the current records.
9. Call app.mainloop() to wait for user interaction.

15.1 Event handling

Each button is connected to its corresponding function using the command argument. The table binds the TreeviewSelect event to fill_form_from_tree(event), allowing a selected record to be loaded back into the form for editing.

16. Input Validation

Validation runs before a record is written. The messages below are represented in the submitted code and are useful for interpreting the behaviour during a demonstration.

Validation case	Message or behaviour
Required fields	All fields are required!
Numeric conversion	Roll Number, Age and Marks must be valid numbers!
Age range	Please enter a valid age.
Subject marks range	Marks must be between 0 and 100.
Duplicate roll on add	Student with this Roll Number already exists!
Update roll not found	Roll Number not found in database!
Delete input invalid	Please enter a valid Roll Number to delete.
Delete roll not found	Roll Number not found!
Missing import headings	Missing required columns: ...
Invalid / missing imported values	CSV contains invalid or missing data in required fields.
Duplicate imported rolls	Duplicate Roll Numbers found: ...
Import error	Failed to import CSV: ...
No data for analytics	No data available.
No data for chart	No data to plot.

16.1 Order of checks

The form is read first. Empty fields are rejected, numeric fields are converted, age and marks ranges are tested, and the record operation checks whether the roll number exists or is duplicated. CSV import adds a column-structure check before accepting records.

17. Analytics Pipeline

The analytics buttons and chart buttons follow a similar sequence: the requested operation is identified, the stored CSV file is read, empty-data conditions are checked, the matching expression or chart logic is evaluated, and the result is displayed.

Input condition	Behaviour
Records are present	The selected summary is computed and shown in a message box, or a chart is opened in a Matplotlib window.
No stored records	Analytics shows “No data available.” and chart actions show “No data to plot.”
Search filter is active	Analytics reads the stored file; its results are not narrowed by the currently filtered table view.
A record is changed	The updated file is read when analytics or chart operations are requested.

18. Dataset Documentation

> SAMPLE / DUMMY DATA — FOR DEMONSTRATION ONLY. The twenty records below are artificial test records, not records of real students.

Property	Value
Number of records	20
Fields per record	13: ten entered fields and three calculated fields
Class levels	Class 11: 10 records; Class 12: 10 records
Sections in sample	Class 11: A2, B4, C2, D2; Class 12: A4, B2, C2, D2
Percentage range	38.75% to 94.50%
Grade distribution	A+: 4; A: 8; B: 2; C: 2; D: 3; F: 1
Roll numbers	101 to 120 with no duplicate rolls
Completeness	260 cells (20 records × 13 fields) filled in the demonstration dataset

18.1 Subject-mark ranges

Subject	Lowest	Highest	90+	Below 40	Average
English	40	95	5	0	77.15
Physics	38	95	5	1	77.05
Chemistry	42	96	6	0	78.65
Mathematics	35	94	5	1	76.90

18.2 Checks satisfied by the sample file

- The file has the thirteen columns in the order expected by the program.
- Roll numbers 101–120 are unique.
- Every subject mark is between 0 and 100.
- The percentage is the total of four subject marks divided by four.
- The grade corresponds to the threshold ladder used by calculate_grade().
- The dataset contains no missing cells.

18.3 Where the file is used

After import, the sample records populate the record table and provide the input for the dashboard, ten analytics operations and three charts. The same rows are reproduced in the next section so that the sample statistics can be traced to their source.

19. Complete Dataset — Student.csv

All twenty records are listed below. To keep the print layout readable on A4 paper, the fields are shown in two sequential tables with Roll Number as the link between them. Together, the tables cover all thirteen columns.

Roll No.	Name	Class	Section	Gender	Age
101	Aarav Sharma	12	A	Male	17
102	Priya Patel	12	A	Female	17
103	Rohan Gupta	12	B	Male	18
104	Anjali Singh	12	B	Female	17
105	Vikram Mehta	11	A	Male	16
106	Sneha Reddy	11	A	Female	16
107	Arjun Kumar	11	B	Male	17
108	Meera Joshi	11	B	Female	16
109	Karan Verma	12	C	Male	18
110	Neha Agarwal	12	C	Female	17
111	Aditya Nair	11	C	Male	16
112	Kavya Iyer	11	C	Female	16
113	Rahul Shah	12	D	Male	17
114	Pooja Menon	12	D	Female	17
115	Siddharth Rao	11	D	Male	16
116	Tanvi Desai	11	D	Female	16
117	Nikhil Bansal	12	A	Male	18
118	Divya Kapoor	12	A	Female	17
119	Manish Tiwari	11	B	Male	16
120	Riya Malhotra	11	B	Female	16

Table 19A. Student identity and basic details.

Roll No.	English	Physics	Chemistry	Mathematics	Total	Percent	Grade
101	85	92	88	90	355	88.75	A
102	78	85	82	79	324	81.00	A
103	92	95	90	93	370	92.50	A+
104	70	65	72	68	275	68.75	C
105	88	91	89	87	355	88.75	A
106	95	93	96	94	378	94.50	A+
107	45	50	48	52	195	48.75	D
108	82	79	81	80	322	80.50	A
109	60	58	62	55	235	58.75	D
110	90	88	92	89	359	89.75	A
111	75	72	78	74	299	74.75	B
112	88	85	90	87	350	87.50	A
113	55	60	58	62	235	58.75	D

Roll No.	English	Physics	Chemistry	Mathematics	Total	Percent	Grade
114	83	80	85	82	330	82.50	A
115	91	89	93	90	363	90.75	A+
116	68	70	65	67	270	67.50	C
117	40	38	42	35	155	38.75	F
118	79	76	80	77	312	78.00	B
119	86	84	88	85	343	85.75	A
120	93	91	94	92	370	92.50	A+

Table 19B. Marks and the three fields calculated by the program. These records are synthetic sample data.

20. Results Computed from the Sample Dataset

Measure	Value
Total students	20
Average percentage	77.44%
Pass percentage	95.00%
Topper	Sneha Reddy
Highest percentage	94.50%
Lowest scorer	Nikhil Bansal
Lowest percentage	38.75%

20.1 Class averages

Class	Average percentage	Records
Class 11	81.12%	10
Class 12	73.75%	10

20.2 Subject averages

Subject	Average mark
English	77.15
Physics	77.05
Chemistry	78.65
Mathematics	76.90

20.3 Grade distribution

Grade	Students	Share
A+	4	20.0%
A	8	40.0%
B	2	10.0%

Grade	Students	Share
C	2	10.0%
D	3	15.0%
F	1	5.0%

20.4 Top five and bottom five

Top five students:

Roll	Name	Class	Percent	Grade
106	Sneha Reddy	11	94.50%	A+
103	Rohan Gupta	12	92.50%	A+
120	Riya Malhotra	11	92.50%	A+
115	Siddharth Rao	11	90.75%	A+
110	Neha Agarwal	12	89.75%	A

Bottom five students:

Roll	Name	Class	Percent	Grade
117	Nikhil Bansal	12	38.75%	F
107	Arjun Kumar	11	48.75%	D
109	Karan Verma	12	58.75%	D
113	Rahul Shah	12	58.75%	D
116	Tanvi Desai	11	67.50%	C

> Interpretation: the demonstration dataset contains 20 records, with a mean percentage of 77.44%. Nineteen students have a grade other than F, giving a pass percentage of 95.00%. Sneha Reddy has the highest percentage (94.50%), while Nikhil Bansal has the lowest (38.75%).

21. Application Outputs

The program presents results through the dashboard cards, message boxes, record table and separate chart windows. The illustrations in this section are based on the strings and widget setup in the source code; the dashboard image is a reconstruction rather than a direct screenshot of a live session.

Output channel	Produced by	What the user sees
KPI cards	update_dashboard()	Student count, average percentage, topper and pass percentage.
Message boxes	messagebox.showinfo / showwarning / showerror	Success messages, validation errors, confirmations or analytics results.
Record table	refresh_table()	All stored records or the rows that match a search.
Chart windows	plot_graph()	A bar or pie chart in a Matplotlib window.

21.1 Representative messages

Type	Title / message
Success	Student Added Successfully!
Warning	All fields are required!
Error	Student with this Roll Number already exists!
Confirm delete	Are you sure you want to delete this student?
Import success	Imported 20 students successfully!
Empty chart	No data to plot.

21.2 Example search results

A search for the text “ar” matches student names containing those letters, such as Aarav Sharma and Arjun Kumar. The following is a small illustrative excerpt from the demonstration dataset.

Roll	Name	Class	Section	Percent	Grade
101	Aarav Sharma	12	A	88.75%	A
107	Arjun Kumar	11	B	48.75%	D
109	Karan Verma	12	C	58.75%	D
110	Neha Agarwal	12	C	89.75%	A

21.3 Dashboard values for the sample file

Total students	Average percentage	Topper name	Pass percentage
20	77.44%	Sneha Reddy	95.00%

22. Test Cases

The following cases form the verification checklist for a live demonstration. Expected results are derived from the validations and operations documented in the source; run them in the application before treating them as observed pass/fail outcomes.

Test	Action	Expected result
1	Add a valid student	Record is appended; success message appears; table and dashboard refresh.
2	Add a duplicate roll number	Duplicate-roll error appears; no duplicate record is added.
3	Submit the form with a blank field	Warning appears; record is not saved.
4	Enter marks above 100, below 0 or as text	Range or numeric-conversion error appears.
5	Enter an age of 0, above 100 or non-numeric text	Age validation message appears.
6	Update an existing student	Stored row is replaced; derived values are recalculated.
7	Update a roll number that does not exist	A roll-not-found message appears.
8	Delete a record and confirm	Confirmation appears first; the selected row is removed.

Test	Action	Expected result
9	Delete using a non-numeric roll number	A validation message appears.
10	Import a valid CSV file	Accepted rows are appended and the import count is reported.
11	Import a file missing required columns	A missing-column error appears; the import is rejected.
12	Import a file with duplicate rolls	A duplicate-roll error appears; the import is rejected.
13	Search a partial name or roll number	Only matching rows are shown; table dashboard values refresh.
14	Press an analytics button	A message box displays the requested summary.
15	Press a chart button	A Matplotlib window displays the requested chart.

22.1 Empty-file behaviour

After all rows are removed, analytics should report “No data available.” and chart actions should report “No data to plot.” The dashboard should show zero values and N/A for the topper name. Confirm these conditions during the live demonstration.

23. Limitations and Future Enhancements

The following are the boundaries of the submitted implementation. The enhancements are proposals only and are not described as features that already exist.

Current limitation	Possible future enhancement
The CSV file is the only data store.	Move to SQLite or MySQL if the project requires stronger data constraints or larger datasets.
There is no sign-in or role-based access.	Add authentication and separate read/write permissions.
Attendance, fees and internal assessment are not included.	Extend the data model with additional school modules if required.
The program does not export printable report cards.	Add export of reports or charts to PDF or image files.
Analytics use the complete stored dataset even when the table is filtered.	Add class-wise or filtered analytics.
The program is run from Python rather than a packaged installer.	Package it as a desktop executable for easier distribution.
Validation feedback uses dialogs.	Highlight the specific field that needs correction.

24. Conclusion

The Student Management System stores student academic records in one CSV file, calculates total marks, percentages and grades, and provides a convenient way to add, update, delete, import and search records. The dashboard and analytics make common questions easier to answer, while the charts present trends in a visual form.

The project combines Python programming, a graphical user interface, file handling, Pandas data manipulation and Matplotlib visualisation. The sample dataset demonstrates how data moves from storage to calculations, summary values and charts. The system remains deliberately small and offline,

and its limitations are listed separately so that proposed future features are not confused with the current implementation.

For a successful demonstration, the program should be run with the supplied Student.csv file, the operations should be tested using the checklist in Section 22, and the student should be prepared to explain the validation rules, grading thresholds, DataFrame operations and three charts.

25. Learning Outcomes

Area	What the project practises
Python fundamentals	Variables, data types, conditions, loops, functions, return values, dictionaries and formatted strings.
File handling	Reading and writing a CSV file, using a stable file path and dealing with an empty or unreadable file.
Pandas and data handling	Creating DataFrames, selecting rows and columns, appending and updating records, filtering data and summarising columns.
Graphical user interface	Creating widgets, binding commands to buttons, using StringVar, filling a Treeview and responding to selection events.
Data visualisation	Creating bar charts and a pie chart, and setting titles and axis labels.
Validation and error handling	Checking required fields and numeric ranges before writing; showing useful error messages.
Testing and documentation	Designing test cases, tracing each button to its function and documenting how to run the project.

26. Bibliography

Publisher	Reference	Website	Use in this project
Python Software Foundation	The Python Tutorial	https://docs.python.org/3/tutorial/	Python syntax, functions, data structures and formatted strings.
Python Software Foundation	Tkinter - Python interface to Tcl/Tk	https://docs.python.org/3/library/tkinter.html	Window creation, widgets, variables and message boxes.
Python Software Foundation	tkinter.ttk - themed widgets	https://docs.python.org/3/library/tkinter.ttk.html	Treeview table and scrollbars.
Python Software Foundation	csv - CSV file reading and writing	https://docs.python.org/3/library/csv.html	CSV concepts and file storage context.
Pandas development team	Pandas documentation	https://pandas.pydata.org/docs/	DataFrames, reading and writing tabular data and summary operations.
Matplotlib development team	Pyplot tutorial	https://matplotlib.org/stable/tutorials/pyplot.html	Creating bar charts, pie charts, titles and axes.
CBSE	Academic resources and Informatics Practices curriculum	https://cbseacademic.nic.in/	Curriculum context for the Class XII Informatics Practices project.

Accessed for programming and documentation background. The project content and sample outputs are based on the submitted application and demonstration dataset.

Appendix A — Complete Source Code

The following 539 source lines are reconstructed from the complete source-code appendix in the previous project report. This code is presented as a single-column listing for legibility; line numbers are omitted from the printed code so it can be copied without removing numbering. The source compiles as Python syntax. Verify the runtime behaviour on the target computer before the viva.

File name: Project Program.py | Source lines: 539 | Functions: 15

A.1 Setup and configuration

```
import os
import tkinter as tk
from tkinter import ttk, messagebox
import customtkinter as ctk
import pandas as pd
import matplotlib.pyplot as plt

# ========================================
# SETUP AND CONFIGURATION
# ========================================

ctk.set_appearance_mode("Dark")
ctk.set_default_color_theme("blue")

FILE_NAME = os.path.join(os.path.dirname(__file__), "students.csv")
COLUMNS = [
    "Roll Number",
    "Name",
    "Class",
    "Section",
    "Gender",
    "Age",
    "English",
    "Physics",
    "Chemistry",
    "Mathematics",
    "Total Marks",
    "Percentage",
    "Grade",
]

if not os.path.exists(FILE_NAME):
    df = pd.DataFrame(columns=COLUMNS)
    df.to_csv(FILE_NAME, index=False)
```

A.2 Helper functions

```
app = ctk.CTk()
app.title("Student Management System")
app.geometry("1400x800")

# ========================================
# HELPER FUNCTIONS
# ========================================


def calculate_grade(percentage):
    if percentage >= 90:
        return "A+"
    elif percentage >= 80:
        return "A"
    elif percentage >= 70:
        return "B"
    elif percentage >= 60:
        return "C"
    elif percentage >= 40:
        return "D"
    else:
        return "F"
```

```
def clear_form():
    roll_var.set("")
    name_var.set("")
    class_var.set("12")
    section_var.set("A")
    gender_var.set("Male")
    age_var.set("")
    eng_var.set("")
    phy_var.set("")
    chem_var.set("")
    math_var.set("")


def refresh_table(search_df=None):
    for row in tree.get_children():
        tree.delete(row)

    try:
        if search_df is not None:
            df = search_df
        else:
            df = pd.read_csv(FILE_NAME)

        for _, row in df.iterrows():
            tree.insert("", "end", values=list(row))

        update_dashboard(df)
    except Exception as e:
        messagebox.showerror("Error", f"Could not load data: {e}")


def update_dashboard(df):
    if df.empty:
        kpi_total.configure(text="0")
        kpi_avg.configure(text="0.0%")
        kpi_topper.configure(text="N/A")
        kpi_pass.configure(text="0.0%")
        return

    total_students = len(df)
    avg_pct = df["Percentage"].mean()
    topper_name = df.loc[df["Percentage"].idxmax()]["Name"]
    pass_count = len(df[df["Grade"] != "F"])
    pass_pct = (pass_count / total_students) * 100

    kpi_total.configure(text=str(total_students))
    kpi_avg.configure(text=f"{avg_pct:.2f}%")
    kpi_topper.configure(text=str(topper_name))
    kpi_pass.configure(text=f"{pass_pct:.2f}%")


def get_form_data():
    try:
        roll = roll_var.get().strip()
        name = name_var.get().strip()
        cls = class_var.get()
        sec = section_var.get()
        gen = gender_var.get()
        age = age_var.get().strip()
        eng = eng_var.get().strip()
        phy = phy_var.get().strip()
        chem = chem_var.get().strip()
        math = math_var.get().strip()

        if not all([roll, name, cls, sec, gen, age, eng, phy, chem, math]):
            messagebox.showwarning("Warning", "All fields are required!")
            return None

        roll = int(roll)
        age = int(age)
        eng = float(eng)
```

```
        phy = float(phy)
        chem = float(chem)
        math = float(math)

        if age <= 0 or age > 100:
            messagebox.showerror("Error", "Please enter a valid age.")
            return None

        for mark in [eng, phy, chem, math]:
            if mark < 0 or mark > 100:
                messagebox.showerror("Error", "Marks must be between 0 and 100.")
                return None

        total = eng + phy + chem + math
        percentage = total / 4.0
        grade = calculate_grade(percentage)

        return {
            "Roll Number": roll,
            "Name": name,
            "Class": cls,
            "Section": sec,
            "Gender": gen,
            "Age": age,
            "English": eng,
            "Physics": phy,
            "Chemistry": chem,
            "Mathematics": math,
            "Total Marks": total,
            "Percentage": percentage,
            "Grade": grade,
        }
    except ValueError:
        messagebox.showerror("Error", "Roll Number, Age and Marks must be valid numbers!")
        return None


# ========================================
# CRUD OPERATIONS
# ========================================
```

A.3 Record operations and CSV import

```
def add_student():
    data = get_form_data()
    if data:
        df = pd.read_csv(FILE_NAME)

        if data["Roll Number"] in df["Roll Number"].values:
            messagebox.showerror("Error", "Student with this Roll Number already exists!")
            return

        df = pd.concat([df, pd.DataFrame([data])], ignore_index=True)
        df.to_csv(FILE_NAME, index=False)

        messagebox.showinfo("Success", "Student Added Successfully!")
        clear_form()
        refresh_table()


def update_student():
    data = get_form_data()
    if data:
        df = pd.read_csv(FILE_NAME)
        roll = data["Roll Number"]

        if roll not in df["Roll Number"].values:
            messagebox.showerror("Error", "Roll Number not found in database!")
            return

        df.loc[df["Roll Number"] == roll, list(data.keys())] = list(data.values())
```

```
        df.to_csv(FILE_NAME, index=False)

        messagebox.showinfo("Success", "Student Updated Successfully!")
        refresh_table()


def delete_student():
    try:
        roll = int(roll_var.get().strip())
        df = pd.read_csv(FILE_NAME)

        if roll not in df["Roll Number"].values:
            messagebox.showerror("Error", "Roll Number not found!")
            return

        confirm = messagebox.askyesno("Confirm Delete", "Are you sure you want to delete this student?")
        if confirm:
            df = df[df["Roll Number"] != roll]
            df.to_csv(FILE_NAME, index=False)
            messagebox.showinfo("Success", "Student Deleted Successfully!")
            clear_form()
            refresh_table()
    except ValueError:
        messagebox.showerror("Error", "Please enter a valid Roll Number to delete.")


def import_csv():
    file_path = tk.filedialog.askopenfilename(
        title="Select CSV File",
        filetypes=[("CSV Files", "*.csv"), ("All Files", "*.*")]
    )
    if not file_path:
        return

    try:
        import_df = pd.read_csv(file_path)

        missing_cols = set(COLUMNS) - set(import_df.columns)
        if missing_cols:
            messagebox.showerror("Error", f"Missing required columns: {', '.join(missing_cols)}")
            return

        import_df = import_df[COLUMNS]

        for col in ["Roll Number", "Age"]:
            import_df[col] = pd.to_numeric(import_df[col], errors="coerce")
        for col in ["English", "Physics", "Chemistry", "Mathematics", "Total Marks", "Percentage"]:
            import_df[col] = pd.to_numeric(import_df[col], errors="coerce")

        if import_df.isnull().any().any():
            messagebox.showerror("Error", "CSV contains invalid or missing data in required fields.")
            return

        existing_df = pd.read_csv(FILE_NAME)
        duplicate_rolls = set(import_df["Roll Number"]).intersection(set(existing_df["Roll Number"]))
        if duplicate_rolls:
            messagebox.showerror("Error", f"Duplicate Roll Numbers found: {', '.join(map(str, duplicate_rolls))}")
            return

        combined_df = pd.concat([existing_df, import_df], ignore_index=True)
        combined_df.to_csv(FILE_NAME, index=False)

        messagebox.showinfo("Success", f"Imported {len(import_df)} students successfully!")
        refresh_table()
    except Exception as e:
        messagebox.showerror("Error", f"Failed to import CSV: {e}")
```

A.4 Search, analytics and chart functions

```
def search_student():
    query = search_var.get().strip().lower()
```

```
    if not query:
        refresh_table()
        return

    df = pd.read_csv(FILE_NAME)
    filtered_df = df[
        df["Roll Number"].astype(str).str.contains(query) |
        df["Name"].str.lower().str.contains(query)
    ]
    refresh_table(filtered_df)


def fill_form_from_tree(event):
    selected_item = tree.selection()
    if selected_item:
        values = tree.item(selected_item[0], "values")
        if values:
            roll_var.set(values[0])
            name_var.set(values[1])
            class_var.set(values[2])
            section_var.set(values[3])
            gender_var.set(values[4])
            age_var.set(values[5])
            eng_var.set(values[6])
            phy_var.set(values[7])
            chem_var.set(values[8])
            math_var.set(values[9])


# ========================================
# ANALYTICS & GRAPHS FUNCTIONS
# ========================================


def show_analytics(action):
    df = pd.read_csv(FILE_NAME)
    if df.empty:
        messagebox.showinfo("Info", "No data available.")
        return

    result = ""
    if action == "Class Average":
        result = df.groupby("Class") ["Percentage"].mean().to_string()
    elif action == "Topper":
        topper = df.loc[df["Percentage"].idxmax()]
        result = f"Name: {topper['Name']} \nPercentage: {topper['Percentage']}%\nClass: {topper['Class']}"
    elif action == "Lowest Scorer":
        lowest = df.loc[df["Percentage"].idxmin()]
        result = f"Name: {lowest['Name']} \nPercentage: {lowest['Percentage']}%\nClass: {lowest['Class']}"
    elif action == "Top 5":
        top5 = df.nlargest(5, "Percentage") [["Name", "Class", "Percentage"]]
        result = top5.to_string(index=False)
    elif action == "Bottom 5":
        bot5 = df.nsmallest(5, "Percentage") [["Name", "Class", "Percentage"]]
        result = bot5.to_string(index=False)
    elif action == "Subject Avg":
        subs = df[["English", "Physics", "Chemistry", "Mathematics"]].mean()
        result = subs.to_string()
    elif action == "Pass Count":
        count = len(df[df["Grade"] != "F"])
        result = f"Total Passed: {count}"
    elif action == "Fail Count":
        count = len(df[df["Grade"] == "F"])
        result = f"Total Failed: {count}"
    elif action == "Pass Pct":
        count = len(df[df["Grade"] != "F"])
        pct = (count / len(df)) * 100
        result = f"Pass Percentage: {pct:.2f}%"
    elif action == "Grade Dist":
        dist = df["Grade"].value_counts()
        result = dist.to_string()

    messagebox.showinfo(action, result)
```

```
def plot_graph(graph_type):
    df = pd.read_csv(FILE_NAME)
    if df.empty:
        messagebox.showinfo("Info", "No data to plot.")
        return

    plt.figure(figsize=(8, 5))

    if graph_type == "Top Students":
        top_students = df.nlargest(5, "Percentage")
        plt.bar(top_students["Name"], top_students["Percentage"], color=["blue", "green", "red", "purple", "orange"])
        plt.title("Top 5 Students by Percentage")
        plt.xlabel("Student Name")
        plt.ylabel("Percentage (%)")
    elif graph_type == "Subject Average":
        averages = df[["English", "Physics", "Chemistry", "Mathematics"]].mean()
        plt.bar(averages.index, averages.values, color=["skyblue", "lightgreen", "salmon", "plum"])
        plt.title("Class Average in Each Subject")
        plt.xlabel("Subjects")
        plt.ylabel("Average Marks")
    elif graph_type == "Grade Distribution":
        grade_counts = df["Grade"].value_counts()
        plt.pie(grade_counts.values, labels=grade_counts.index, autopct="%1.1f%%", startangle=140)
        plt.title("Grade Distribution of Students")

    plt.tight_layout()
    plt.show()


# ========================================
# GUI LAYOUT DESIGN
# ========================================

header_frame = ctk.CTkFrame(app, fg_color="transparent")
header_frame.pack(fill="x", pady=10)

ctk.CTkLabel(header_frame, text="STUDENT MANAGEMENT SYSTEM", font=("Arial", 32, "bold")).pack()
ctk.CTkLabel(header_frame, text="CBSE Informatics Practices Project", font=("Arial", 16)).pack()

kpi_frame = ctk.CTkFrame(app)
kpi_frame.pack(fill="x", padx=20, pady=10)
```

A.5 GUI layout and application start

```
def create_kpi_card(parent, title):
    card = ctk.CTkFrame(parent, fg_color="#2b2b2b", corner_radius=10)
    card.pack(side="left", fill="both", expand=True, padx=10, pady=10)
    ctk.CTkLabel(card, text=title, font=("Arial", 14)).pack(pady=(10, 0))
    val_label = ctk.CTkLabel(card, text="-", font=("Arial", 24, "bold"), text_color="#1f6aa5")
    val_label.pack(pady=(5, 10))
    return val_label


kpi_total = create_kpi_card(kpi_frame, "Total Students")
kpi_avg = create_kpi_card(kpi_frame, "Average Percentage")
kpi_topper = create_kpi_card(kpi_frame, "Topper Name")
kpi_pass = create_kpi_card(kpi_frame, "Pass Percentage")

main_frame = ctk.CTkFrame(app, fg_color="transparent")
main_frame.pack(fill="both", expand=True, padx=20, pady=10)

form_frame = ctk.CTkFrame(main_frame, width=350)
form_frame.pack(side="left", fill="y", padx=(0, 10))

ctk.CTkLabel(form_frame, text="Student Details Form", font=("Arial", 20, "bold")).pack(pady=15)

roll_var = tk.StringVar()
name_var = tk.StringVar()
class_var = tk.StringVar(value="12")
```

```
section_var = tk.StringVar(value="A")
gender_var = tk.StringVar(value="Male")
age_var = tk.StringVar()
eng_var = tk.StringVar()
phy_var = tk.StringVar()
chem_var = tk.StringVar()
math_var = tk.StringVar()


def make_entry(parent, label, variable, is_combo=False, combo_values=None):
    frame = ctk.CTkFrame(parent, fg_color="transparent")
    frame.pack(fill="x", padx=20, pady=5)
    ctk.CTkLabel(frame, text=label, width=100, anchor="w").pack(side="left")
    if is_combo:
        comp = ctk.CTkComboBox(frame, variable=variable, values=combo_values, width=180)
    else:
        comp = ctk.CTkEntry(frame, textvariable=variable, width=180)
    comp.pack(side="right")


make_entry(form_frame, "Roll Number:", roll_var)
make_entry(form_frame, "Student Name:", name_var)
make_entry(form_frame, "Class:", class_var, True, ["11", "12"])
make_entry(form_frame, "Section:", section_var, True, ["A", "B", "C", "D"])
make_entry(form_frame, "Gender:", gender_var, True, ["Male", "Female", "Other"])
make_entry(form_frame, "Age:", age_var)
make_entry(form_frame, "English Marks:", eng_var)
make_entry(form_frame, "Physics Marks:", phy_var)
make_entry(form_frame, "Chemistry Marks:", chem_var)
make_entry(form_frame, "Maths Marks:", math_var)

btn_frame = ctk.CTkFrame(form_frame, fg_color="transparent")
btn_frame.pack(pady=20)

ctk.CTkButton(btn_frame, text="Add", width=80, command=add_student).grid(row=0, column=0, padx=5, pady=5)
ctk.CTkButton(btn_frame, text="Update", width=80, command=update_student).grid(row=0, column=1, padx=5, pady=5)
ctk.CTkButton(btn_frame, text="Delete", width=80, fg_color="red", hover_color="darkred", command=delete_student).grid(row=0,
column=2, padx=5, pady=5)
ctk.CTkButton(btn_frame, text="Import CSV", width=80, fg_color="#2e7d32", hover_color="#1b5e20", command=import_csv).grid(row=0,
column=3, padx=5, pady=5)
ctk.CTkButton(btn_frame, text="Clear Fields", width=260, fg_color="gray", command=clear_form).grid(row=1, column=0, columnspan=4,
pady=5)

right_frame = ctk.CTkFrame(main_frame, fg_color="transparent")
right_frame.pack(side="right", fill="both", expand=True)

search_frame = ctk.CTkFrame(right_frame, fg_color="transparent")
search_frame.pack(fill="x", pady=(0, 10))

search_var = tk.StringVar()
ctk.CTkEntry(search_frame, textvariable=search_var, placeholder_text="Search by Name or Roll Number", width=300).pack(side="left",
padx=(0, 10))
ctk.CTkButton(search_frame, text="Search", width=100, command=search_student).pack(side="left", padx=5)
ctk.CTkButton(search_frame, text="View All", width=100, fg_color="gray", command=lambda: [search_var.set(""),
refresh_table()]).pack(side="left", padx=5)

style = ttk.Style()
style.theme_use("default")
style.configure("Treeview", background="#2b2b2b", foreground="white", rowheight=25, fieldbackground="#2b2b2b", borderwidth=0)
style.map("Treeview", background=[("selected", "#1f6aa5")])
style.configure("Treeview.Heading", background="#1f1f1f", foreground="white", font=("Arial", 10, "bold"))

tree_frame = ctk.CTkFrame(right_frame)
tree_frame.pack(fill="both", expand=True)

scroll_y = ttk.Scrollbar(tree_frame, orient="vertical")
scroll_x = ttk.Scrollbar(tree_frame, orient="horizontal")

tree = ttk.Treeview(tree_frame, columns=COLUMNS, show="headings", yscrollcommand=scroll_y.set, xscrollcommand=scroll_x.set)
scroll_y.config(command=tree.yview)
scroll_x.config(command=tree.xview)

scroll_y.pack(side="right", fill="y")
```

```
scroll_x.pack(side="bottom", fill="x")
tree.pack(fill="both", expand=True)

for col in COLUMNS:
    tree.heading(col, text=col)
    tree.column(col, width=90, anchor="center")

tree.bind("<<TreeviewSelect>>", fill_form_from_tree)

bottom_panel = ctk.CTkFrame(right_frame, fg_color="transparent")
bottom_panel.pack(fill="x", pady=10)

analytics_lbl = ctk.CTkLabel(bottom_panel, text="Data Analytics & Reports", font=("Arial", 16, "bold"))
analytics_lbl.pack(pady=(0, 5))

an_frame = ctk.CTkFrame(bottom_panel, fg_color="transparent")
an_frame.pack()

analytics_list = [
    "Class Average",
    "Topper",
    "Lowest Scorer",
    "Top 5",
    "Bottom 5",
    "Subject Avg",
    "Pass Count",
    "Fail Count",
    "Pass Pct",
    "Grade Dist",
]

row_val = 0
col_val = 0
for item in analytics_list:
    ctk.CTkButton(
        an_frame,
        text=item,
        width=110,
        fg_color="#3a3a3a",
        hover_color="#4a4a4a",
        command=lambda txt=item: show_analytics(txt),
    ).grid(row=row_val, column=col_val, padx=5, pady=5)
    col_val += 1
    if col_val > 4:
        col_val = 0
        row_val += 1

graph_frame = ctk.CTkFrame(bottom_panel, fg_color="transparent")
graph_frame.pack(pady=10)

ctk.CTkButton(graph_frame, text=" Top Students Chart", width=180, fg_color="#1f6aa5", command=lambda: plot_graph("Top
Students")).pack(side="left", padx=10)
ctk.CTkButton(graph_frame, text=" Subject Average Chart", width=180, fg_color="#1f6aa5", command=lambda: plot_graph("Subject
Average")).pack(side="left", padx=10)
ctk.CTkButton(graph_frame, text=" Grade Distribution Pie", width=180, fg_color="#1f6aa5", command=lambda: plot_graph("Grade
Distribution")).pack(side="left", padx=10)

refresh_table()

if __name__ == "__main__":
    app.mainloop()
```

Appendix B — Data File Reference

The main file students.csv has thirteen fixed columns in the same order shown in the COLUMNS list of Project Program.py. The final three fields are derived values rather than entries typed directly by the user.

#	Field	Written by	Rule
1	Roll Number	User	Whole number; unique among stored records.
2	Name	User	Text; not blank.
3	Class	Combo box	11 or 12.
4	Section	Combo box	A, B, C or D.
5	Gender	Combo box	Male, Female or Other.
6	Age	User	Integer greater than 0 and not more than 100.
7	English	User	Subject marks from 0 to 100.
8	Physics	User	Subject marks from 0 to 100.
9	Chemistry	User	Subject marks from 0 to 100.
10	Mathematics	User	Subject marks from 0 to 100.
11	Total Marks	Program	Sum of English, Physics, Chemistry and Mathematics.
12	Percentage	Program	Total marks divided by four.
13	Grade	Program	A+, A, B, C, D or F according to calculate_grade().

B.1 Example rows from Student.csv

Roll	Name	Class	Sec	Eng	Phy	Chem	Math	Total	%	Grade
101	Aarav Sharma	12	A	85	92	88	90	355	88.75	A
102	Priya Patel	12	A	78	85	82	79	324	81.00	A
103	Rohan Gupta	12	B	92	95	90	93	370	92.50	A+
104	Anjali Singh	12	B	70	65	72	68	275	68.75	C

B.2 Loading the demonstration data

1. Place Student.csv in the folder containing Project Program.py.
2. Start the program and press Import CSV.
3. Select Student.csv in the file chooser.
4. Review the import result and verify that twenty records appear in the table.

B.3 Data integrity rules

- A roll number should not occur more than once.
- Required form fields must not be empty.
- Age must be between 1 and 100, and each subject mark must be between 0 and 100.
- Total marks, percentage and grade are calculated from the four subject marks.